

## Assignment #3 — System Design & Implementation Plan

MTAT.03.229 Enterprise System Integration (Spring 2026)

**System:** QuickBite — Food Delivery System

**Group 7:** Alfa-Kilo | Sierra-Lima | Elephant-Yankee | Mike-Alfa

### 1. Introduction

This document builds on the QuickBite design from Assignment 1 and Assignment 2. It describes a detailed plan for the implementation phase of the project. It covers:

- the full microservices architecture;
- the REST APIs and data models of the services we will build;
- the main end-to-end workflows;
- the synchronous and asynchronous integration mechanisms;
- the final implementation tasks for each team member.

The business architecture is kept separate from the technical implementation view. Figure 1a stays business-only, while Figure 1b adds the client, gateway, messaging, and database details needed for Assignment 3.

### 2. Complete System Architecture

#### 2.1 Business Services (from Assignment 1)

QuickBite has **eight business microservices**. We keep all eight in the architecture so that the full business logic is still visible. Only a part of them will be implemented in the project.

| # | Service                     | Main Responsibility                                                                                                               | Key Data Owned             |
|---|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| 1 | <b>User Service</b>         | Handles sign-up, sign-in, and user profiles for customers, drivers, restaurant owners, and admins.                                | Users, roles, addresses    |
| 2 | <b>Restaurant Service</b>   | Handles restaurant sign-up, profile, location, opening hours, and open/closed status.                                             | Restaurants                |
| 3 | <b>Menu Service</b>         | Publishes each restaurant's catalogue: items, prices, and availability.                                                           | Menu items                 |
| 4 | <b>Order Service</b>        | Owns the order life cycle (Placed -> Paid -> Confirmed -> Prepared -> Ready -> PickedUp -> Delivered) and drives order placement. | Orders, order items        |
| 5 | <b>Payment Service</b>      | Processes payments and refunds for orders (simulated).                                                                            | Payments, transactions     |
| 6 | <b>Delivery Service</b>     | Creates delivery tasks, matches drivers, and tracks delivery progress.                                                            | Delivery tasks             |
| 7 | <b>Notification Service</b> | Sends messages to users when a status changes.                                                                                    | Notifications              |
| 8 | <b>Review Service</b>       | Collects ratings and reviews after delivery and keeps the average rating per restaurant.                                          | Reviews, rating aggregates |

## 2.2 Service Relationships (business level)

- **Order Service** is the main coordinator. It calls User, Restaurant, Menu, Payment, and Delivery Services to place an order. It also tells Notification Service about status changes.
- **Delivery Service** reads driver profiles from User Service and exchanges status updates with Order Service. It also tells Notification Service about driver assignment and new delivery states.
- **Review Service** checks with Order Service (a review is only allowed after Delivered) and with User Service (to link the review to the reviewer).
- **Menu Service** depends on Restaurant Service, because menus belong to a restaurant.
- **Notification Service** is called asynchronously by any service that needs to inform the user about a status change.
- **User Service** is used by every other service for identity and access checks.

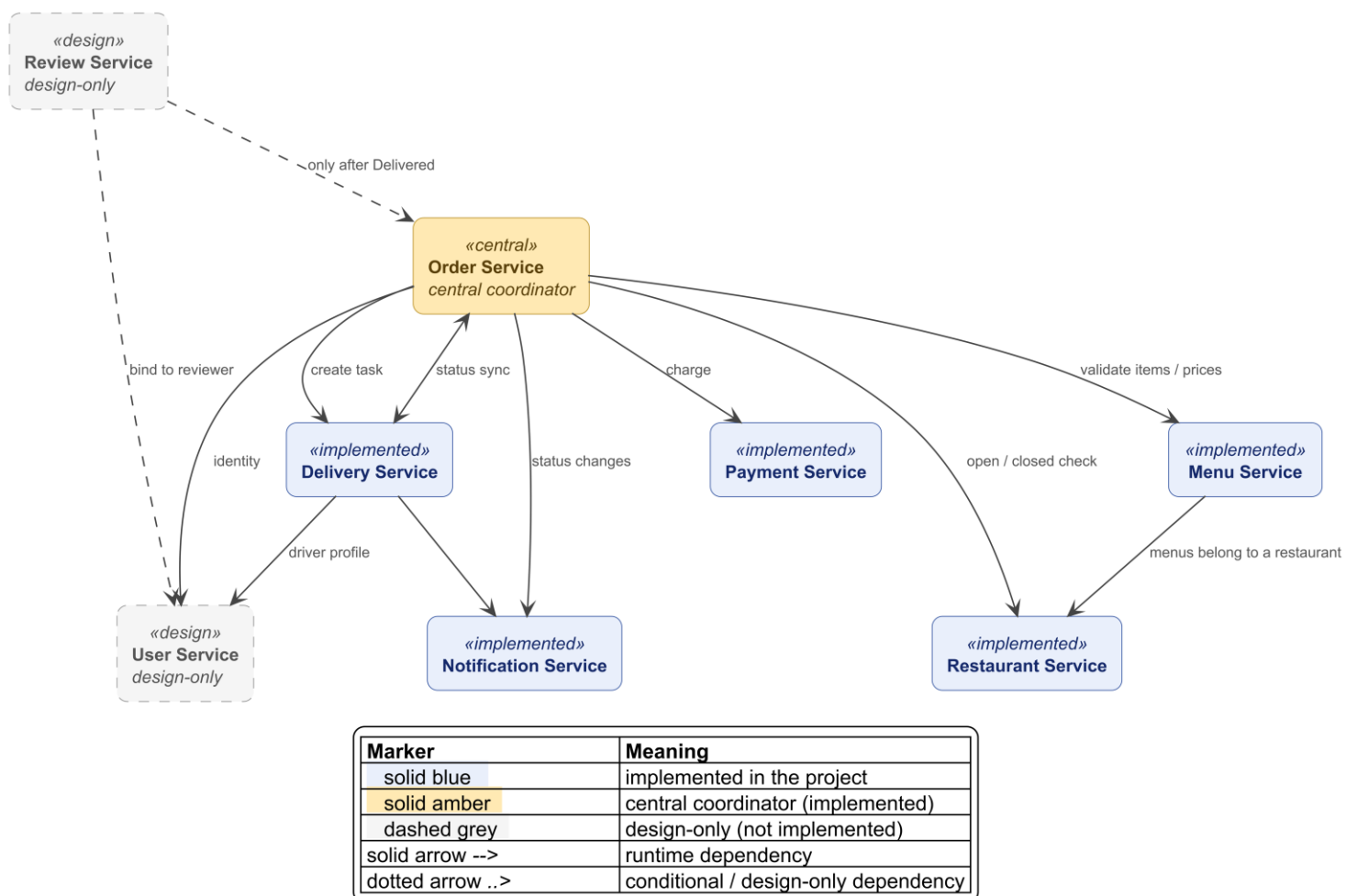


Figure 1a — Complete business architecture diagram

## 2.3 Implementation Architecture (technical view)

Assignment 3 also needs a technical view of the runtime setup, so we keep that view separate from the business architecture instead of mixing both concerns into one figure. The implementation architecture shows the client entry point, the API Gateway, the implemented services, the Kafka topics, and one database per implemented business service.

This technical view is adapted from the Lecture 10 reference architecture, but simplified to match the actual project scope:

- the client calls only the API Gateway;
- the API Gateway routes requests to the implemented REST services and forwards the stubbed X-User-Id header used in place of the design-only User Service;
- the Order Service performs synchronous REST calls to Restaurant, Menu, Payment, and Delivery during Workflow W1;
- Payment, Delivery, and Order publish domain events to Kafka, while Order and Notification consume the relevant topics;
- we do **not** add a separate service-discovery server, because static service configuration is enough at the scale of this project.

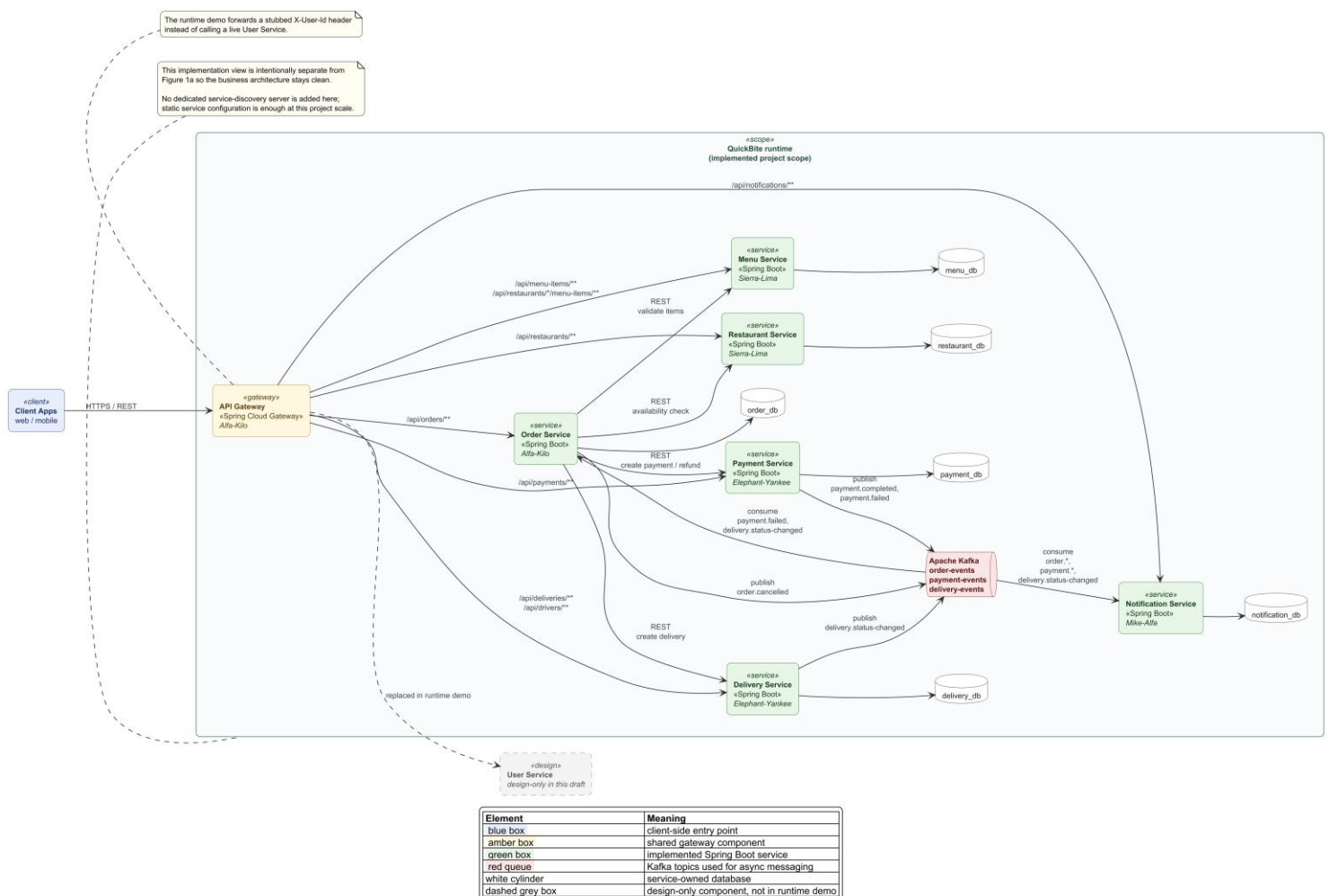


Figure 1b — Implementation architecture diagram

## 2.4 Subset Implemented in the Project

To keep the scope realistic, we implement only a part of the system. Each student implements **two components**. These can be two business services, or one business service plus one shared integration/resilience component.

| Status                                      | Services / Components                                           |
|---------------------------------------------|-----------------------------------------------------------------|
| Implemented (business services)             | Order, Restaurant, Menu, Payment, Delivery, Notification        |
| Implemented (shared integration/resilience) | API Gateway (Alfa-Kilo), Event Broker configuration (Mike-Alfa) |
| Design only (not implemented)               | User Service, Review Service                                    |

**Why this choice.** User Service and Review Service stay in the design because login and post-delivery reviews are important parts of the business. But neither is needed to show the two required interaction styles. Replacing them with an API Gateway and an event broker lets us show both synchronous integration (gateway + REST) and asynchronous integration (event broker), while still keeping six business services in the implementation.

This allocation also stays aligned with Section 7 of the assignment brief: each student implements at most two components, and at most one original service may be replaced by a shared integration or resilience component. For that reason, the User Service remains design-only in the assessed implementation scope instead of being added on top of Order Service plus API Gateway.

For the two services that are not implemented:

- the implemented services use a stubbed identity (an X-User-Id header set by the API Gateway) in place of the User Service;
- no service needs to call the Review Service at run time.

### 3. Service APIs

We only describe the services that will be implemented in the project. Each service stays within the 5–8 endpoints guideline from the assignment brief. All paths go through the API Gateway (see §3.7).

#### 3.1 Order Service (*Owner: Alfa-Kilo*)

| Endpoint                | Method | Description                                                                                                    |
|-------------------------|--------|----------------------------------------------------------------------------------------------------------------|
| /orders                 | POST   | Place a new order. Validates the restaurant and the menu items, starts payment, and creates the delivery task. |
| /orders/{id}            | GET    | Get the details and current status of an order.                                                                |
| /orders/{id}            | DELETE | Cancel a pending order. Only allowed before the restaurant accepts it.                                         |
| /orders/{id}/status     | PATCH  | Internal status change. Called by Payment and Delivery through event handlers.                                 |
| /orders?customerId={id} | GET    | List the orders of a given customer (order history).                                                           |
| /orders/{id}/items      | GET    | List the items of a given order.                                                                               |

**3.2 Restaurant Service (Owner: Sierra-Lima)**

| Endpoint                       | Method | Description                                                                                 |
|--------------------------------|--------|---------------------------------------------------------------------------------------------|
| /restaurants                   | POST   | Register a new restaurant profile.                                                          |
| /restaurants/{id}              | GET    | Get a restaurant profile.                                                                   |
| /restaurants/{id}              | PUT    | Update the restaurant profile (hours, location, details).                                   |
| /restaurants/{id}/status       | PATCH  | Change the open/closed status.                                                              |
| /restaurants                   | GET    | Search or list restaurants (filters: city, open).                                           |
| /restaurants/{id}/availability | GET    | Quick check to see if a restaurant is open. Called by Order Service during order placement. |

**3.3 Menu Service (Owner: Sierra-Lima)**

| Endpoint                      | Method | Description                                                                                                           |
|-------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------|
| /restaurants/{rid}/menu-items | POST   | Add a new menu item to a restaurant.                                                                                  |
| /restaurants/{rid}/menu-items | GET    | List the menu items of a given restaurant.                                                                            |
| /menu-items/{id}              | GET    | Get a single menu item (price and availability).                                                                      |
| /menu-items/{id}              | PUT    | Update a menu item.                                                                                                   |
| /menu-items/{id}              | DELETE | Remove a menu item.                                                                                                   |
| /menu-items/validate          | POST   | Validate a list of {menuItemid, quantity} and return the unit prices. Called by Order Service during order placement. |

**3.4 Payment Service (Owner: Elephant-Yankee)**

| Endpoint                     | Method | Description                                            |
|------------------------------|--------|--------------------------------------------------------|
| /payments                    | POST   | Start a payment charge for an order.                   |
| /payments/{id}               | GET    | Get a payment and its status.                          |
| /payments/by-order/{orderId} | GET    | Get the payment that belongs to a given order.         |
| /payments/{id}/refund        | POST   | Issue a refund for a cancelled or failed order.        |
| /payments/{id}/transactions  | GET    | List the transactions (charge or refund) of a payment. |

The Payment Service also publishes the asynchronous events *payment.completed* and *payment.failed* (see §6).

### 3.5 Delivery Service (Owner: *Elephant-Yankee*)

| Endpoint                       | Method | Description                                                  |
|--------------------------------|--------|--------------------------------------------------------------|
| /deliveries                    | POST   | Create a delivery task for a paid order.                     |
| /deliveries/{id}               | GET    | Get a delivery task.                                         |
| /deliveries/{id}/assign        | POST   | Assign an available driver to a delivery task.               |
| /deliveries/{id}/status        | PATCH  | Driver updates the status (Assigned → PickedUp → Delivered). |
| /deliveries/by-order/{orderId} | GET    | Get the delivery task that belongs to a given order.         |
| /drivers/{id}/active           | GET    | Get the active delivery of a given driver.                   |

The Delivery Service also publishes the asynchronous event `delivery.status-changed` (see §6).

### 3.6 Notification Service (Owner: *Mike-Alfa*)

| Endpoint                    | Method | Description                                                      |
|-----------------------------|--------|------------------------------------------------------------------|
| /notifications              | GET    | List the notifications of the current user.                      |
| /notifications/{id}         | GET    | Get a single notification.                                       |
| /notifications/unread-count | GET    | Get the number of unread notifications for the current user.     |
| /notifications/{id}/read    | PATCH  | Mark one notification as read.                                   |
| /notifications/read-all     | PATCH  | Mark all notifications of the current user as read.              |
| /notifications/send         | POST   | Internal send endpoint. Used for testing and admin actions only. |

The Notification Service is mainly event-driven. It consumes `order.*`, `payment.*`, and `delivery.status-changed` events (see §6).

### 3.7 API Gateway (Owner: *Alfa-Kilo* — *shared integration component*)

The API Gateway is not a REST service on its own. It is the single entry point for clients and it handles cross-cutting concerns. We use Spring Cloud Gateway.

| Route prefix                                         | Target service       |
|------------------------------------------------------|----------------------|
| /api/orders/**                                       | Order Service        |
| /api/restaurants/**                                  | Restaurant Service   |
| /api/menu-items/**, /api/restaurants/*/menu-items/** | Menu Service         |
| /api/payments/**                                     | Payment Service      |
| /api/deliveries/**, /api/drivers/**                  | Delivery Service     |
| /api/notifications/**                                | Notification Service |

The gateway also does:

- request routing and path rewriting;
- CORS handling in one place;
- simple rate limiting;
- forwarding a stubbed X-User-Id header to downstream services (used instead of the missing User Service).

### 3.8 Messaging / Event Broker (*Owner: Mike-Alfa — shared integration component*)

This is also not a REST service. It provides the messaging layer used by the asynchronous workflows in §5 and §6. We use **Apache Kafka**, because it has good support in Spring Boot (through Spring for Apache Kafka) and is easy to run as one container in our demo setup. The topics, producers, and consumers are listed in §6.2.

## 4. Service Data Models

Each implemented service has its own database. No service reads another service's tables directly. Cross-service links are stored as ID fields (marked with \*), the same rule as in Assignment 2.

### 4.1 Order Service DB (*Alfa-Kilo*)

#### Entities

- **Order** (*aggregate root*) — orderId (PK), customerId\*, restaurantId\*, status (Placed | Confirmed | Paid | Prepared | Ready | PickedUp | Delivered | Cancelled), placedAt, totalAmount, deliveryStreet, deliveryCity, deliveryPostalCode.
- **OrderItem** — orderItemId (PK), orderId (FK), menuItemId\*, name, unitPrice, quantity.

**Relationships.** One Order has many OrderItem rows.

### 4.2 Restaurant Service DB (*Sierra-Lima*)

- **Restaurant** (*aggregate root*) — restaurantId (PK), ownerId\*, name, address, city, latitude, longitude, operatingHours, isOpen.

### 4.3 Menu Service DB (*Sierra-Lima*)

- **MenuItem** (*aggregate root*) — menuItemId (PK), restaurantId\*, name, description, priceAmount, priceCurrency, category, isAvailable.

**Relationships.** Logical: many MenuItem rows belong to one restaurant (restaurantId\*).

### 4.4 Payment Service DB (*Elephant-Yankee*)

- **Payment** (*aggregate root*) — paymentId (PK), orderId\*, amount, currency, status (Pending | Completed | Failed | Refunded), processedAt.

- **Transaction** — transactionId (PK), paymentId (FK), type (charge | refund), amount, occurredAt.

**Relationships.** One Payment has many Transaction rows.

#### 4.5 Delivery Service DB (*Elephant-Yankee*)

- **DeliveryTask** (*aggregate root*) — deliveryId (PK), orderId\*, driverId\*, pickupStreet, pickupCity, dropoffStreet, dropoffCity, status (Pending | Assigned | PickedUp | Delivered), assignedAt.

#### 4.6 Notification Service DB (*Mike-Alfa*)

- **Notification** (*aggregate root*) — notificationId (PK), recipientId\*, channel (email | sms | push), message, sentAt, status (Queued | Sent | Failed | Read).

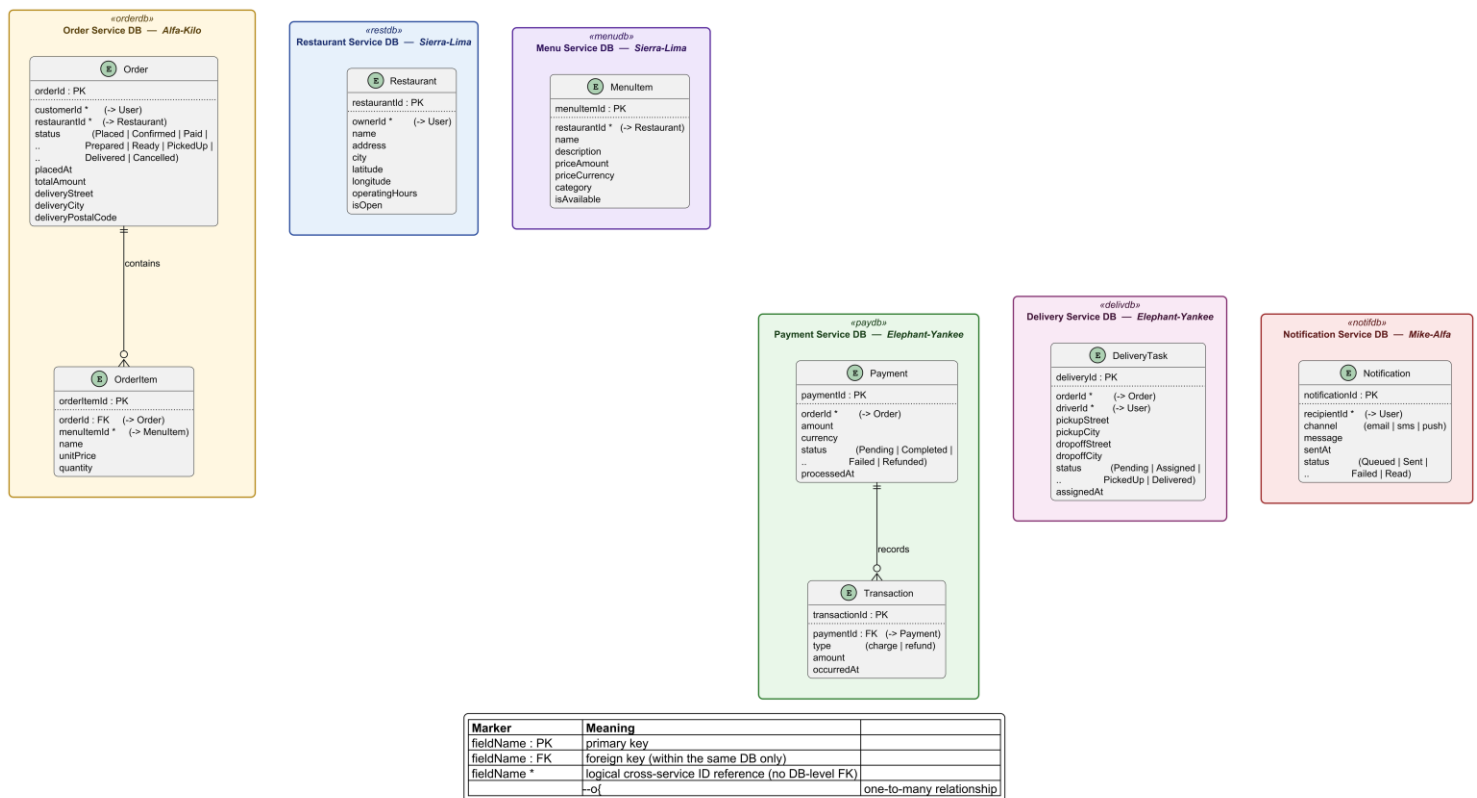


Figure 2 — ER diagram for each implemented service

## 5. System Workflows

This section presents the main end-to-end workflows. Workflows W1 and W2 are selected for implementation in the project. Together, they cover **both** required interaction styles:

- W1 shows synchronous REST-based coordination;
- W2 shows asynchronous event-driven integration.

W3 is an extra asynchronous workflow that goes together with W1. W4 is kept at the design level only.

## 5.1 W1 — Place Order (*synchronous, implemented*)

**Trigger.** A customer submits an order from the client app.

**Services involved.** API Gateway, Order Service, Restaurant Service, Menu Service, Payment Service, Delivery Service.

### Steps.

1. Client → API Gateway → **Order Service**: POST /orders with {customerId, restaurantId, items[], deliveryAddress}.
2. Order Service → **Restaurant Service**: GET /restaurants/{id}/availability to confirm that the restaurant is open.
3. Order Service → **Menu Service**: POST /menu-items/validate with the list of items. This checks that each item exists and is available, returns the unit prices, and lets the Order Service compute totalAmount.
4. Order Service saves the order with status = Placed in its own database (Order and OrderItem tables).
5. Order Service → **Payment Service**: POST /payments with {orderId, amount}. The charge runs synchronously.
6. If the payment result is Completed, Order Service → **Delivery Service**: POST /deliveries with {orderId, pickupAddress, dropoffAddress}. The order status is then updated to Paid, and then to Confirmed.
7. Order Service returns 201 Created to the client, with the order id and the first status.

**Tables updated.** orders, order\_items (Order DB); payments, transactions (Payment DB); delivery\_tasks (Delivery DB).

### Failure handling.

- If step 3 fails, the Order Service returns 422.
- If step 5 fails, the order is set to Cancelled and no delivery task is created.
- If step 6 fails, the Order Service runs a compensating action: a refund through POST /payments/{id}/refund.

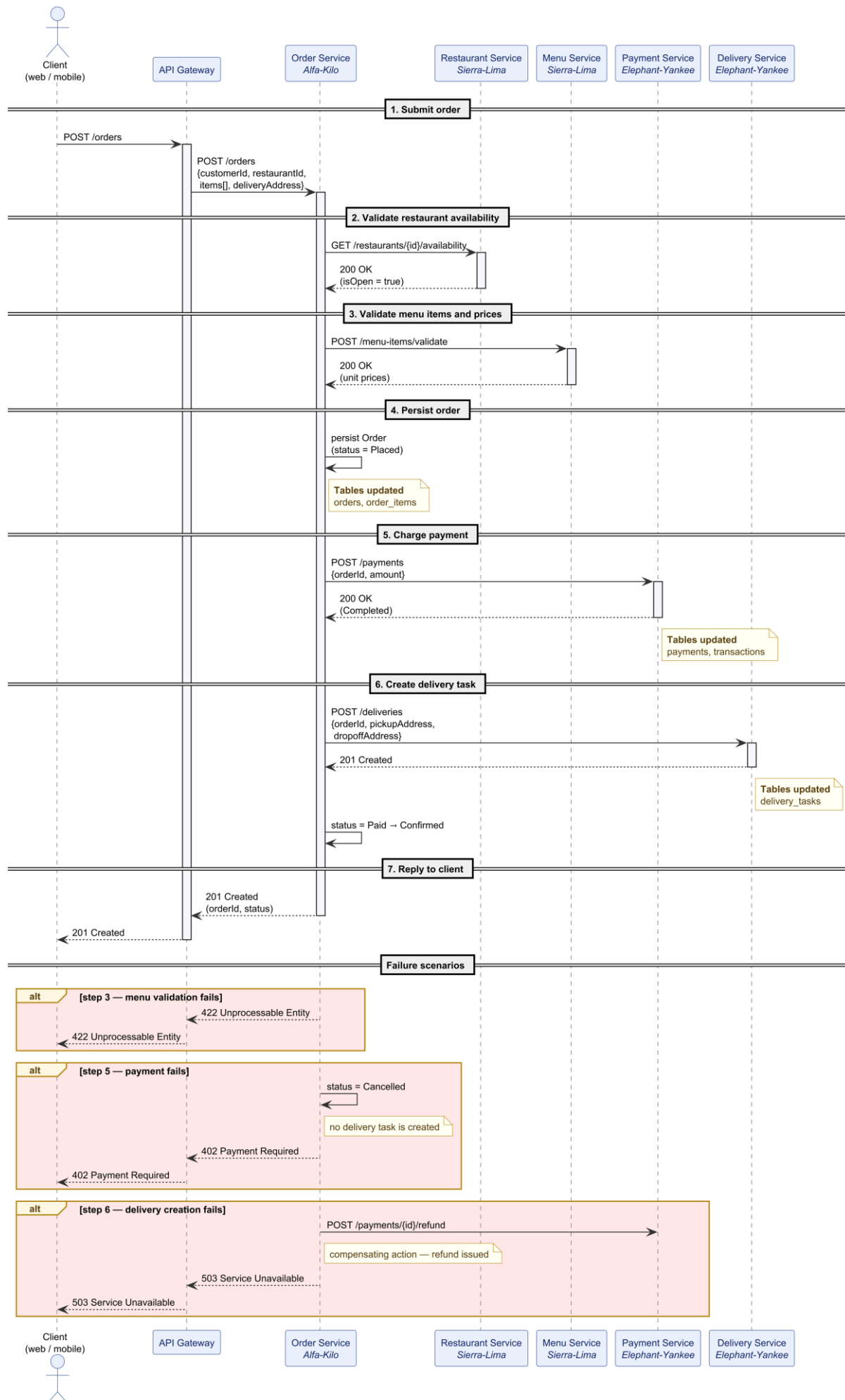


Figure 3 — Sequence diagram of Workflow W1

## 5.2 W2 — Delivery Progress & Notifications (*asynchronous, implemented*)

**Trigger.** The driver updates the delivery status in the Delivery Service.

**Services involved.** Delivery Service (producer), Order Service (consumer), Notification Service (consumer), Event Broker.

### Steps.

1. Driver client → **Delivery Service**: PATCH `/deliveries/{id}/status` with the new value (PickedUp or Delivered).
2. Delivery Service saves the new status in its own database.
3. Delivery Service **publishes** the event `delivery.status-changed` to the topic `delivery-events` with the payload `{deliveryId, orderId, status, occurredAt}`.
4. **Order Service** (consumer) updates the matching order status (PickedUp → PickedUp; Delivered → Delivered) in its own database.
5. **Notification Service** (consumer) creates a user-facing notification for the customer and the driver, saves it, and records the send attempt.

**Tables updated.** `delivery_tasks` (Delivery DB); `orders` (Order DB); `notifications` (Notification DB).

## 5.3 W3 — Payment Completed → Order Confirmation (*asynchronous, implemented as complement to W1*)

After step 5 of W1, the Payment Service also publishes `payment.completed` or `payment.failed` to the topic `payment-events`. The Notification Service consumes these events and tells the customer whether the payment was accepted or not. This keeps the notification logic out of the synchronous order-placement path.

## 5.4 W4 — Submit Review (*design only, not implemented*)

Included for completeness. A customer submits a review for a delivered order. The Review Service checks with the Order Service before saving the review. This workflow is not implemented, because the Review Service is design-only.

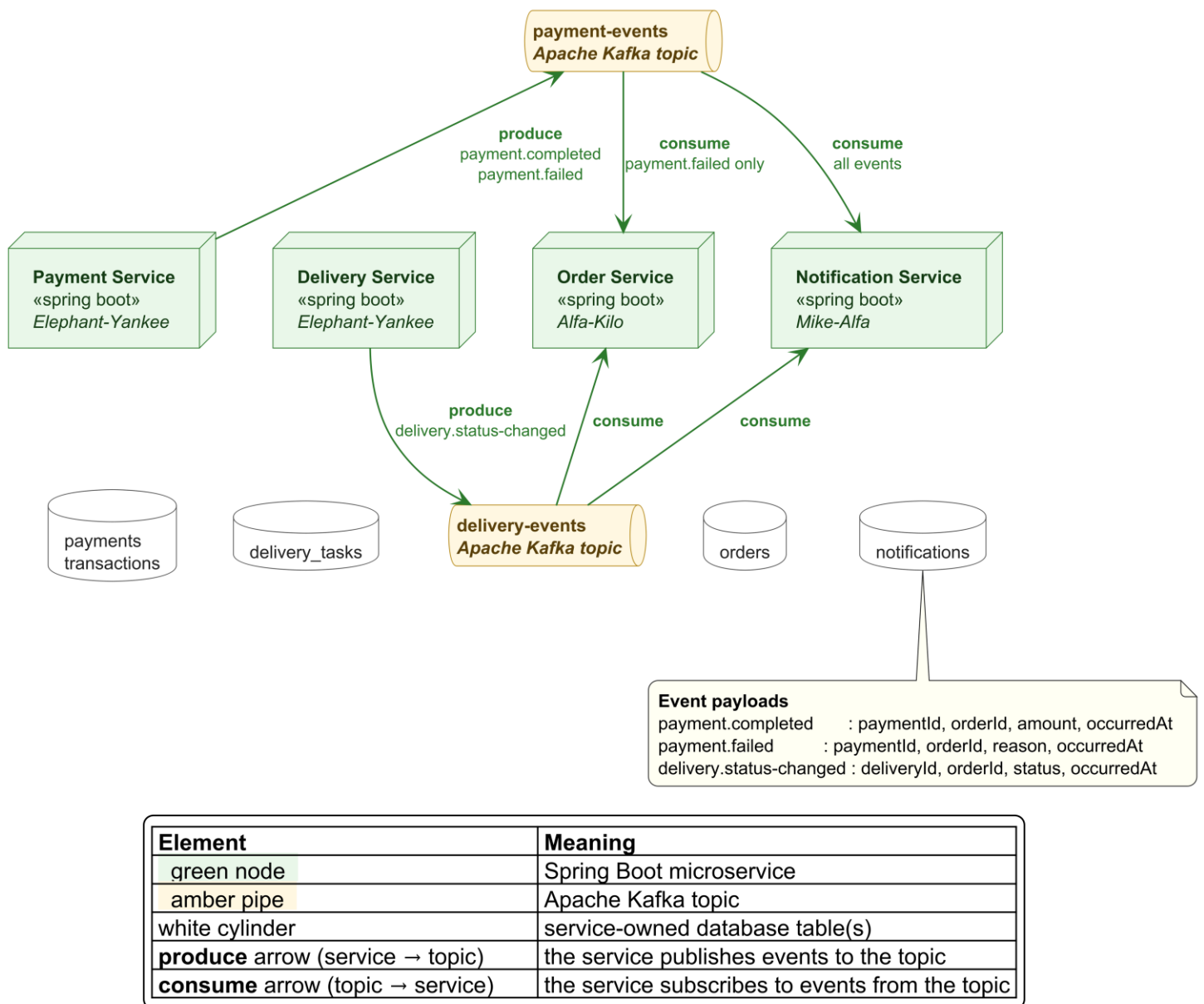


Figure 4 — Event-flow diagram for workflows W2 and W3

## 6. Integration Mechanisms

### 6.1 Synchronous integration (REST)

- **Transport.** HTTP with JSON bodies, through the **API Gateway** (Spring Cloud Gateway). Service-to-service calls inside the system are direct REST calls. Static configuration is enough at the scale of this project, so we do not add a separate service-discovery server.
- **Used in.** W1 (Order ↔ Restaurant, Menu, Payment, Delivery), delivery creation, menu validation.
- **Error handling.** HTTP status codes are mapped to domain errors. The Order Service runs compensating actions (for example, a refund) when a downstream call fails.

### 6.2 Asynchronous integration (events)

An **event broker** carries the domain events. We use **Apache Kafka**, and the broker configuration is owned by **Mike-Alfa** as the shared integration component.

| Topic / Exchange | Event                   | Producer         | Consumer(s)                                         | Payload data                            |
|------------------|-------------------------|------------------|-----------------------------------------------------|-----------------------------------------|
| payment-events   | payment.completed       | Payment Service  | Notification Service                                | paymentId, orderId, amount, occurredAt  |
| payment-events   | payment.failed          | Payment Service  | Notification Service, Order Service                 | paymentId, orderId, reason, occurredAt  |
| delivery-events  | delivery.status-changed | Delivery Service | Order Service, Notification Service                 | deliveryId, orderId, status, occurredAt |
| order-events     | order.cancelled         | Order Service    | Payment Service (runs refund), Notification Service | orderId, reason, occurredAt             |

### Guarantees.

- At-least-once delivery.
- Consumers are idempotent: the event id is used as the key, so a duplicate event is handled only once.
- Failed handlers are routed to a dead-letter queue.

### 6.3 Cross-cutting

- **API Gateway** (Alfa-Kilo) — one entry point for clients, request routing, CORS handling, basic rate limiting, and forwarding of the stubbed X-User-Id header (used instead of the missing User Service).
- **Event Broker configuration** (Mike-Alfa) — declares topics and queues, sets the dead-letter policy, and defines a shared event envelope (id, type, occurredAt, payload).

## 7. Final Implementation Responsibilities

Each student implements **two components**. As allowed by Section 7 of the assignment brief, two students replace one of their original services with a shared integration/resilience component. All four students still keep at least one business service that has a non-trivial REST API, owns its own database, and takes part in at least one implemented workflow.

| Student                | Component 1 (business service)                                                                    | Component 2                                                                                              | Notes                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Alfa-Kilo</b>       | <b>Order Service</b> — REST API (§3.1), DB (§4.1), main coordinator of W1, consumer in W2         | <b>API Gateway</b> (shared integration component, replaces User Service)                                 | User Service stays design-only. Stubbed identity is forwarded by the gateway. |
| <b>Sierra-Lima</b>     | <b>Restaurant Service</b> — REST API (§3.2), DB (§4.2), takes part in W1                          | <b>Menu Service</b> — REST API (§3.3), DB (§4.3), takes part in W1                                       | Both business services are implemented.                                       |
| <b>Elephant-Yankee</b> | <b>Payment Service</b> — REST API (§3.4), DB (§4.4), takes part in W1 and produces payment-events | <b>Delivery Service</b> — REST API (§3.5), DB (§4.5), takes part in W1 and produces delivery-events (W2) | Both business services are implemented.                                       |
| <b>Mike-Alfa</b>       | <b>Notification Service</b> — REST API (§3.6), DB (§4.6), consumer in W2 and W3                   | <b>Event Broker configuration</b> (shared integration component, replaces Review Service)                | Review Service stays design-only.                                             |

### Constraint checks.

- Each student implements exactly two components. ✓
- At most one service per student is replaced by a shared component (Alfa-Kilo and Mike-Alfa each replace one). ✓
- Each student with a replaced service still keeps a business service (Order, Notification) that:
  - exposes a REST API with at least 5 endpoints,
  - owns its own data,
  - takes part in two or more inter-service interactions,
  - takes part in at least one implemented workflow. ✓
- The selected workflows cover both synchronous (W1) and asynchronous (W2, W3) interactions. ✓
- At least six business services are implemented, so the system stays realistic and coherent. ✓

## 8. Summary

The QuickBite implementation plan covers six business microservices plus two shared integration components. They are driven by two end-to-end workflows, which together show both synchronous REST-based coordination and asynchronous event-driven integration. The work is clearly split between the four team members.